Multi-modal Learning Project with Vison Based Models Using CLIP for Connecting Text and Images

Matthew Harper
University of Houston Victoria

Topic Areas:
Transformers Vision Transformers (ViTs) Image Classification Vision-Language Models (VLMs) Multi-Modal Models CLIP Classification Models

Abstract: Deep learning has often been characterized as a type of intelligence developed to mirror the human brain with systems of neurons which are essential in the human thinking process. Where many modern deep learning models have successfully mimicked one form of input such as text, auditory, or vision, a newer sub-field of artificial intelligence, multi-modal learning, seeks to replicate the ability of the human brain by analyzing and processing multiple forms of input to gain understanding and draw insights [2]. Moreover, multi-modal models typically may analyze an image, a textual description or prompt, or an auditory sound bite to perform a task [1]. This report provides a brief introduction of the multi-modal model learning process followed by project exercise.

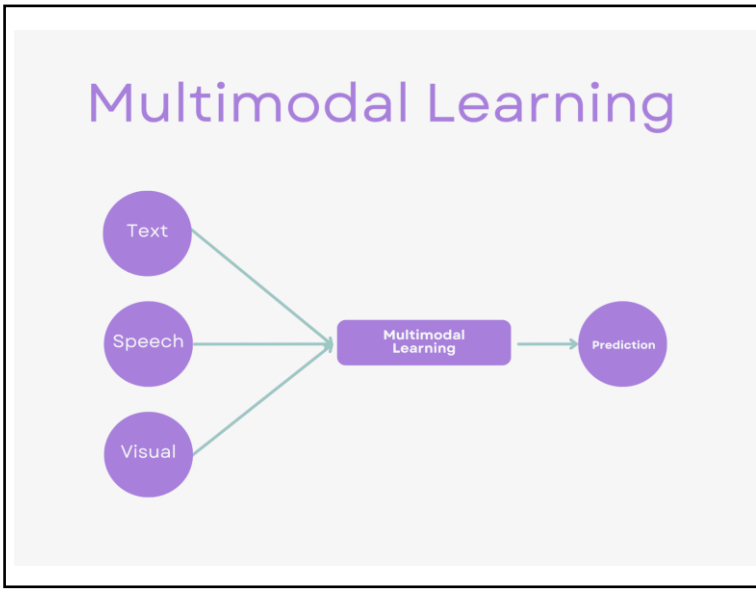


Figure 1 – *Simplistic multi-modal model representation illustrating the multiple sourced sensory input into model processing for predicted outcome [2].*

Introduction

Multi-modal models can be described as machine learning models that are capable of processing information from different modalities or different types of input data simultaneously [1,3]. Where traditional machine learning models typically rely on a single form or modality of input such as text, image, or sound, real-world environments, tasks, and decision making often requires the processing of many forms of input simultaneously. Moreover, multi-modal model learning has an objective to connect different types of input, thereby enabling a model to learn in a comprehensive manner to derive and understand contextually complex representations [1].

Multimodal AI is a more advanced form of AI that can process and analyze multiple types of data (like video, audio, images, and text) simultaneously. This allows it to gain a more comprehensive and nuanced understanding of its surroundings, similar to how humans do. By combining information from different sources, multimodal AI can identify patterns and relationships that might be missed by traditional AI

systems that focus on just one type of data. This enhanced understanding enables multimodal AI to make more accurate and informed decisions, leading to improved performance in various applications [1,5].

In addition, where Generative AI can be thought of as a use of machine learning to generate content such as text, images, and audio often from nothing more than a prompt, multi-modal models are a form of Generative AI which expands these capabilities by processing multiple modalities. This multimodality provides the model with the ability to process and interpret many different sensory modes. Moreover, this also provides a unique ability to generative and compare multiple predictions based on multiple modes and then compare the results, with a higher degree of accuracy and precision based on improved data quality [1, 3].

Traditionally, the combination of models has been and remains a common technique in machine learning which utilizes multiple models to improve the performance of a single model. The underlying principle and hypothesis is that one model's strengths can compensate for another's weakness, resulting in a more accurate and robust prediction. Classical model combination methods often include stacking, bagging, and ensemble techniques which have the following characteristics [2]:

Classical Methods for Combining Data

Ensemble – A modeling technique where multiple diverse base models are utilized to predict an outcome with the objective to reduce generalization error.

Stacking – Stacking enables the ensembling of classification or regression models by "*stacking*" two-layer estimators. The first layer consists of all the baseline models that are used to predict the outputs on the test datasets. The second layer consists of Meta-Classifier or Regressor which takes all the predictions of baseline models as an input and generate new predictions [4].

Bagging – Bagging (or bootstrapping), is a machine learning technique that improves the stability and accuracy of models by creating multiple subsets of the original data with replacement. These subsets are used to train different models, and their predictions are combined to produce a final result. This helps reduce overfitting and improve generalization performance [4].

Clearly, the combining of model data is particularly useful when the individual models have complementary strengths and weaknesses, and oftentimes leads to a more comprehensive understanding of the data set to produce a more precise prediction.

In contrast, with multi-modal models, the objective is to combine the use of information from different modalities to perform a prediction task. This can involve using techniques such as the fusion-based approach, alignment-based approach, or late fusion to create a high-dimensional representation that captures the semantic information of the data from each modality [1,2].

In addition, multimodal learning models commonly produce superior results when different modalities provide complementary information that can improve the accuracy of the prediction. Moreover, multimodal learning techniques enable models to process and analyze data from multiple modalities effectively, providing a more complete and accurate understanding of the underlying data [2].

Applications of Multi-modal Model

Multi-modal models have many applications as a result of its integrated design utilizing many different types of data including text, images, video, and audio. As a result, multi-modal learning has found

applications in a wide array of uses including image captioning, healthcare, robotics, entertainment, and virtual assistants as follows [1]:

Image captioning – Image captioning is the process of generating descriptive text for an image. Multi-modal models, which are capable of processing both visual and textual information, are used to analyze the visual content of the image and generate a coherent textual description that accurately captures the scene depicted. These models combine the strengths of different modalities to provide a more comprehensive understanding of the image and produce more informative and accurate captions [1].

Healthcare – Multi-modal AI systems are used to analyze medical images (like X-rays and MRIs) in conjunction with patient medical records. This combined analysis can assist in diagnosis, saving time and potentially improving the accuracy of medical decisions [1].

Robotics – Robotics and autonomous systems utilize multi-modal AI to interact with their environment, combining visual input (like camera streamed images) with sensory input data (including auditory, touch and distance information). This integration enables these systems to navigate and perform tasks autonomously, reducing human intervention and potentially leading to new applications in various fields such as object manipulation, navigation, and human-robot interaction [1].

Entertainment – Multi-modal AI is revolutionizing content creation systems by enabling the generation of multi-modal content, such as text, images, and videos. For instance, AI can generate engaging and creative text, produce stunning images, and even compose music, expanding the possibilities for content creation and innovation [1].

Virtual Assistants – Multi-modal AI enhances virtual assistants by allowing them to comprehend and respond to both textual and visual information. This advancement enables these assistants to provide more comprehensive and contextually relevant assistance, understanding queries in relation to visual cues and the user's environment [1].

Multi-modal Vision-Language Models

Vision-Language Models (VLMs) are vision based AI multi-modal models which are bridging the gap between visual and textual data models by processing both visual (like images) and textual (like words) information simultaneously. Their primary goal is to enable machines to understand and interact with the world in a more human-like way by connecting visual and textual content. This has applications in various fields, including image captioning, visual question answering, and cross-modal retrieval. VLMs achieve this by combining computer vision and natural language processing techniques [1].

Moreover, VLMs can understand the content of images and generate corresponding text descriptions, answer questions about visual content, and even search for images based on text descriptions. By integrating visual and textual information, VLMs open up new possibilities for AI applications and contribute to the development of more advanced and versatile intelligent systems [1].

VLMs are able to integrate visual and textual information through a multi-step process as follows [1]:

Visual Encoding: The visual input (like an image) is processed using a Vision Encoder, often a convolutional neural network (CNN) or vision transformer, to extract features representing the image's content.

Textual Encoding: The textual input (like a caption or question) is processed using a Text Encoder, typically a transformer, to convert the text into a sequence of numerical vectors.
Cross-Modal Alignment: The visual and textual features are aligned to establish correspondences between the visual elements and the textual descriptions. This allows the model to understand how visual and textual content relate to each other.

Fusion and Output Generation: The aligned features are combined using fusion mechanisms (often transformers) to generate the final output, which can be a text caption, answer to a visual question, or even a new image based on a text prompt.

Overall, the process above allows VLMs to integrate and connect visual and textual information, leading to their ability to perform tasks like image captioning, visual question answering, and text-to-image generation. CLIP or Contrastive Language-Image Pre-training model is an example of how VLMs can integrate multi-modal input and sensory data, particularly image and textual into a versatile vision-language model.

Contrastive Language-Image Pre-training or **CLIP**, originally developed by OpenAI, is a foundational vision-language model that has significantly impacted the vision-language modeling. Initially trained on massive datasets of images and text captions using a contrastive learning approach, the model was able to successfully learn associations between visual and textual content [1].

In addition, CLIP demonstrates versatility with its ability to perform zero-shot image classification tasks, meaning it can apply its learned knowledge to new tasks without requiring specific fine-tuning or specialized training. This makes it a valuable tool for various applications, including image retrieval and captioning, image classification, image search, and text-to-image generation [1].

As our lab will later demonstrate, VLMs integrated with the us of CLIP's key innovations allows multi-modal models to better understand the relationship between text and images without relying on task-specific fine-tuning. This allows it to be more adaptable and efficient for a wide range of applications.

Future of Multi-modal Model Learning
Future of multi-modal AI research is significantly focused on improving the continued integration and understanding of various data types. Some key developments include the development of improved fusion techniques with more effective methods to combine data from a wider range of sensory data sources allowing for improved decision-making. In addition, models are becoming highly capable of processing complex and contradictory information adding to the usefulness and robustness associated with multimodal Generative AI models [1, 5]. As multi-modal learning continues to advance, additional key research areas which are emerging include [1]:

Unified Multi-modal Models: Researchers are working towards developing unified models that can seamlessly integrate and process various modalities (like text, images, and audio) without requiring specialized training for each modality.
Self-supervised and Unsupervised Learning: To reduce reliance on large labeled datasets, efforts are focused on self-supervised and unsupervised learning techniques that can learn from unlabeled data or by using intrinsic tasks within the data itself.

Cross-modal Retrieval and Generation: Techniques for efficient retrieval and generation of information across different modalities are becoming increasingly important. This includes tasks like finding images based on text descriptions or generating videos from text prompts.

Improved Alignment and Fusion Techniques: Enhancing the alignment and fusion of information from different modalities is a critical area. Innovations in attention mechanisms, multi-modal transformers, and other fusion techniques aim to improve the understanding and integration of diverse data.

These research directions are driving the development of more powerful and versatile multi-modal learning systems, with applications in various fields such as natural language processing, computer vision, and robotics.

Opportunities in Multi-modal Learning

The future of multi-modal learning holds immense potential for innovation and positive impact [1]:

Interdisciplinary Research - Multi-modal learning naturally intersects with various fields, fostering collaboration and leading to more robust AI systems.

New Business Models - The technology's ability to provide personalized and context-aware services will create opportunities for new business models and products.

Global Impact - Multi-modal AI can address global challenges like improving education and healthcare access in underserved areas.

Innovation in AI Ethics - The growing influence of multi-modal AI requires responsible development and deployment to ensure public trust and maximize its benefits.

Model Challenges in Multiple Modalities

Integrating multiple modalities presents several challenges [1]:

Data Volume: Collecting and managing large amounts of data from different modalities can be difficult. Insufficient data can hinder model performance.

Computational Complexity: Processing and integrating diverse data is computationally intensive, potentially slowing down training and deployment.

Data Alignment: Ensuring accurate alignment between different modalities (e.g., text and images) is crucial for effective learning.

Limited Datasets: Scarcity of large, annotated multi-modal datasets can limit model training and lead to overfitting.

Decision-Making Complexity: Making decisions based on multiple modalities requires complex reasoning and can make models harder to interpret.

Missing Data: Incomplete data can impair model performance, as all modalities are typically needed for accurate predictions.

Project – Multi-modal Models: Connecting Text and Images

Code Reference: Huggingface. (n.d.). notebooks/examples/image_classification.ipynb at main · huggingface/notebooks. GitHub. https://github.com/huggingface/notebooks/blob/main/examples/image_classification.ipynb

The process for the Transformer Classification:

Step 1 – Load the Python Libraries datasets, evaluate, transforms et.al.
Step 2 – Import & Load CLIP
Step 3 – Load Vision Transformer (ViT)
Step 4 – Verify Multi-Modal VLM Processing
Step 5 – Load Scikit Image & Tokenize Image Descriptions
Step 6 – Assign Root Path & Data Directories - Upload Images
Step 7 – Preprocess Images & Join Tokenized Descriptions
Step 8 – Encode Transformers with Image/Text Features
Step 9 – Visualize & Compare Image Classification Confusion Matrix (colored Heat Map)
Step 10 – Perform Zero-Shot Image Classification
Step 11 – Debugging Steps

Course: COSC 6370 Advanced Topics in AI
Student: Matthew Harper
Assignment: Lab 5 - CLIP: Connecting Text & Images
Reference: https://github.com/openai/CLIP/blob/main/notebooks/Interacting_with_CLIP.ipynb

Step 1 – Load the Python Libraries datasets, transformers, and evaluate

The first of the lab is to import the required libraries.

```
[ ] ! pip install ftfy regex tqdm

Collecting ftfy
  Downloading ftfy-6.2.3-py3-none-any.whl.metadata (7.8 kB)
Requirement already satisfied: regex in /usr/local/lib/python3.10/dist-packages (2024.9.11)
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (4.66.5)
Requirement already satisfied: wcwidth<0.3.0,>=0.2.12 in /usr/local/lib/python3.10/dist-packages
Downloading ftfy-6.2.3-py3-none-any.whl (43 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 43.0/43.0 kB 1.6 MB/s eta 0:00:00
Installing collected packages: ftfy
Successfully installed ftfy-6.2.3

[ ] ! pip install git+https://github.com/openai/CLIP.git
```

Assign the Pre-trained model that we will fine-tune to our dataset with batch size 32.

```
[ ] import numpy as np
    import torch
    from pkg_resources import packaging

    print("Torch version:", torch.__version__)

    Torch version: 2.4.1+cu121
```

Step 2 – Import CLIP and View (print) Available Models

We will be using the Vision Transformer Model B/32.

Loading the Model

```
[ ] import clip

    clip.available_models()

    ['RN50',
     'RN101',
     'RN50x4',
     'RN50x16',
     'RN50x64',
     'ViT-B/32',
     'ViT-B/16',
     'ViT-L/14',
     'ViT-L/14@336px']
```

Step 3 – Assign Comparative Vision Transformer Model B/32

```
model, preprocess = clip.load("ViT-B/32")
model.cuda().eval()
input_resolution = model.visual.input_resolution
context_length = model.context_length
vocab_size = model.vocab_size

print("Model parameters:", f"{np.sum([int(np.prod(p.shape)) for p in model.parameters()]):,}")
print("Input resolution:", input_resolution)
print("Context length:", context_length)
print("Vocab size:", vocab_size)
```

```
100%|┃┃┃┃┃┃┃┃┃┃┃┃┃┃┃┃┃┃┃┃┃┃┃┃┃┃┃┃┃┃┃┃┃┃| 338M/338M [00:05<00:00, 63.5MiB/s]
Model parameters: 151,277,313
Input resolution: 224
Context length: 77
Vocab size: 49408
```

Step 4 – Verify VLM Processing is Loaded and working

call the preprocess method and observe the current configuration with image resizing of 224x224 and normalized pixel values.

Image Preprocessing

```
[ ] preprocess
```

```
Compose(
    Resize(size=224, interpolation=bicubic, max_size=None, antialias=True)
    CenterCrop(size=(224, 224))
    <function _convert_image_to_rgb at 0x7d9b3afeab00>
    ToTensor()
    Normalize(mean=(0.48145466, 0.4578275, 0.40821073), std=(0.26862954, 0.26130
```

Test
the tokenization process with the Hello World string

Text Preprocessing

```
[ ] clip.tokenize("Hello World!")
```

```
tensor([[49406,  3306,  1002,   256, 49407,     0,     0,     0,     0,     0,
             0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
             0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
             0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
             0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
             0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
             0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
```

Step 5 – Load Scikit-Image and image libraries and descriptions to tokenize

Load the required image processing libraries and define the image descriptions to be tokenized with the VLM.

Setting up input images and texts

```
[ ] import os
    import skimage
    from skimage import data
    import requests
    from io import BytesIO
    import IPython.display
    import matplotlib.pyplot as plt
    from PIL import Image
    import numpy as np

    from collections import OrderedDict
    import torch

    %matplotlib inline
    %config InlineBackend.figure_format = 'retina'

    # images in skimage to use and their textual descriptions
    descriptions = {
        "page": "a page of text about segmentation",
        "chelsea": "a facial photo of a tabby cat",
        "astronaut": "a portrait of an astronaut with the American flag",
        "rocket": "a rocket standing on a launchpad",
        "motorcycle_right": "a red motorcycle standing in a garage",
        "camera": "a person looking at a camera on a tripod",
        "horse": "a black-and-white silhouette of a horse",
        "coffee": "a cup of coffee on a saucer"
    }
```

Step 6 – Assign root and data directories and Load the images for local drive use

```
[ ] from google.colab import drive
    drive.mount('/gdrive')
    # the project's folder
    %cd /gdrive/'My Drive'/Lab_Files
```

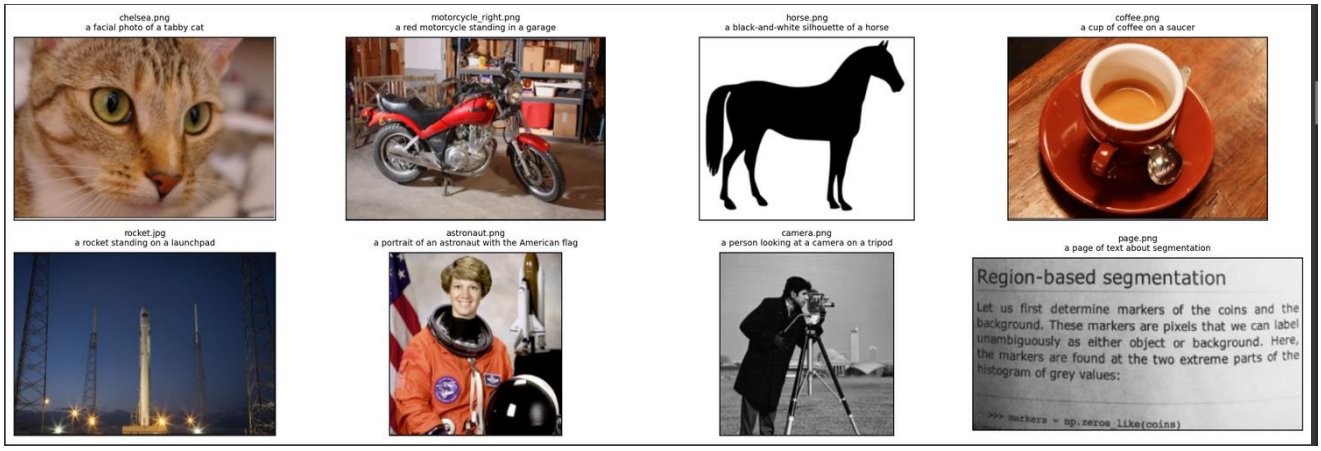
```
Mounted at /gdrive
/gdrive/My Drive/Lab_Files
```

```
[ ]
    root_dir = "/gdrive/MyDrive/"
    data_dir = root_dir + 'Lab_Files'
    print("base directory")
    %ls {data_dir}
```

```
base directory
astronaut.png  chelsea.png  horse.png              page.png
camera.png     coffee.png   motorcycle_right.png   rocket.jpg
```

Step 7 – Preprocess Images and Join Tokenized language descriptions into VLM

```
original_images = []
images = []
texts = []
plt.figure(figsize=(16, 5))
root_dir = "/gdrive/MyDrive/"
data_dir = root_dir + 'Lab_Files'

for filename in [filename for filename in os.listdir(data_dir) if filename.endswith(".png") or filename.endswith(
    name = os.path.splitext(filename)[0]
    if name not in descriptions:
        continue

    image = Image.open(os.path.join(data_dir, filename)).convert("RGB")

    plt.subplot(2, 4, len(images) + 1)
    plt.imshow(image)
    plt.title(f"{filename}\n{descriptions[name]}")
    plt.xticks([])
    plt.yticks([])

    original_images.append(image)
    images.append(preprocess(image))
    texts.append(descriptions[name])

plt.tight_layout()
```



Step 8 – Encode the Image with Text Features as Multi-modal Inputs into the Model

Building features

We normalize the images, tokenize each text input, and run the forward pass of the model to get the image and text features

```
[ ] image_input = torch.tensor(np.stack(images)).cuda()
    text_tokens = clip.tokenize(["This is " + desc for desc in texts]).cuda()
```

```
[ ] with torch.no_grad():
        image_features = model.encode_image(image_input).float()
        text_features = model.encode_text(text_tokens).float()
```

Step 9 – Visualize & Compare the Feature differences of Image and Text by plotting a Confusion Matrix (colored as a heatmap) of the similarities

Begin by normalizing the image and text feature inputs.
Next plot the heat map for each image which compares the image to the prompted textual description.

Calculating cosine similarity

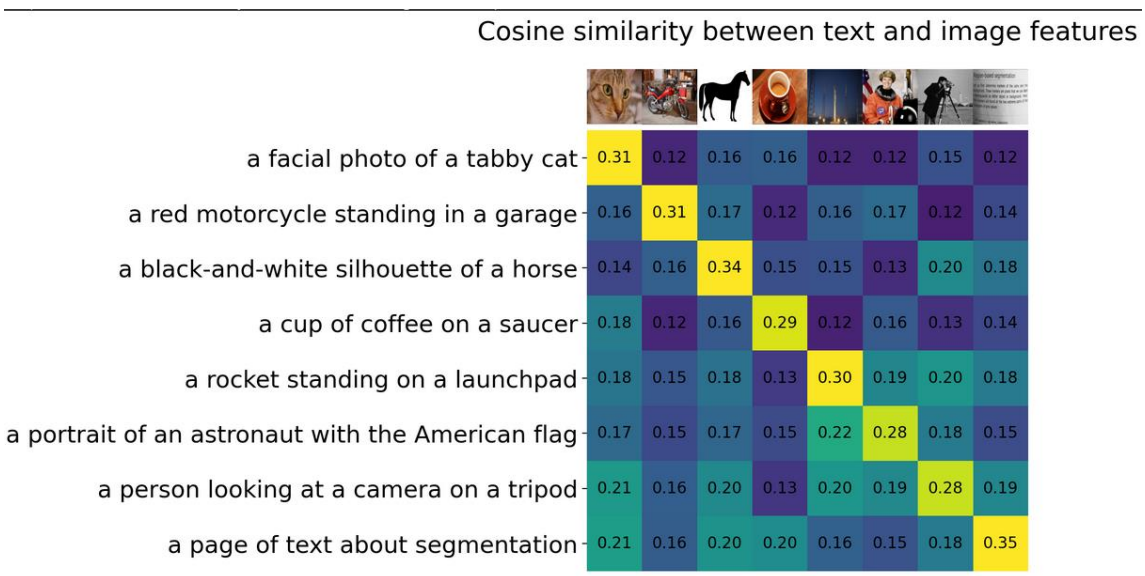
We normalize the features and calculate the dot product of each pair.

```
[ ] image_features /= image_features.norm(dim=-1, keepdim=True)
    text_features /= text_features.cpu().numpy() @ image_features.cpu().numpy().T
```

```
[ ]
    count = len(descriptions)

    plt.figure(figsize=(10, 8))
    plt.imshow(similarity, vmin=0.1, vmax=0.3)
    # plt.colorbar()
    plt.yticks(range(count), texts, fontsize=18)
    plt.xticks([])
    for i, image in enumerate(original_images):
        plt.imshow(image, extent=(i - 0.5, i + 0.5, -1.6, -0.6), origin="lower")
    for x in range(similarity.shape[1]):
        for y in range(similarity.shape[0]):
            plt.text(x, y, f"{similarity[y, x]:.2f}", ha="center", va="center", size=12)

    for side in ["left", "top", "right", "bottom"]:
      plt.gca().spines[side].set_visible(False)

    plt.xlim([-0.5, count - 0.5])
    plt.ylim([count + 0.5, -2])

    plt.title("Cosine similarity between text and image features", size=20)
```

```
Text(0.5, 1.0, 'Cosine similarity between text and image features')
```

Observe the Confusion Matrix - Notice how the diagonal yellow crossing line is similar to confusion matrix.



Step 10 – Perform a Zero-Shot Image Classification Inference with No Fine-Tuning

This step demonstrates the robustness of a VLM multi-modal model

Zero-Shot Image Classification

You can classify images using the cosine similarity (times 100) as the logits to the softmax operation.

```
[ ] from torchvision.datasets import CIFAR100

    cifar100 = CIFAR100(os.path.expanduser("~/.cache"), transform=preprocess, download=True)
```

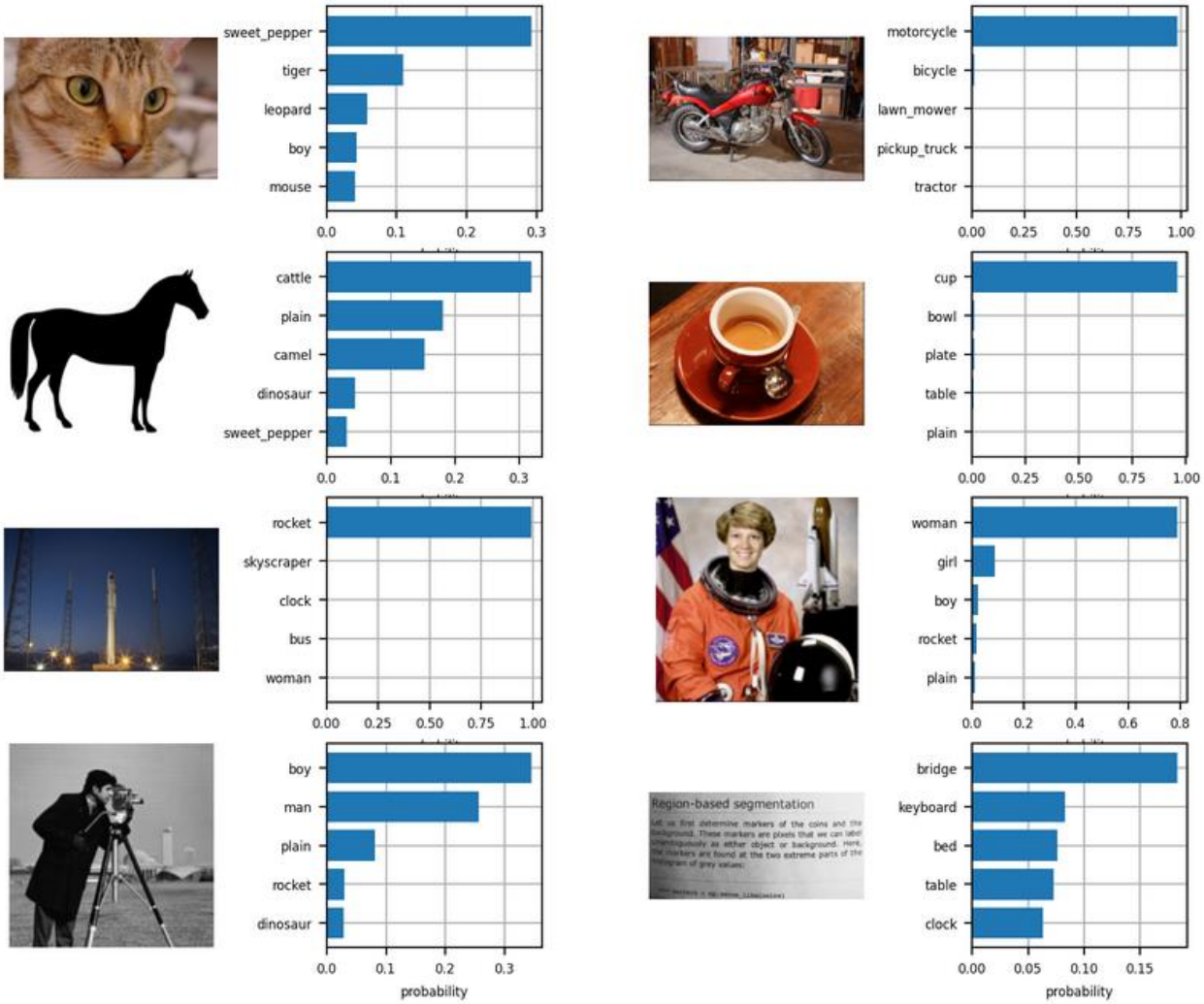
```
Downloading https://www.cs.toronto.edu/~kriz/cifar-100-python.tar.gz to /root/.cache/cifar-100-python.tar.gz
100%|██████████| 169001437/169001437 [00:08<00:00, 19434773.97it/s]
Extracting /root/.cache/cifar-100-python.tar.gz to /root/.cache
```

```
text_descriptions = [f"This is a photo of a {label}" for label in cifar100.classes]
text_tokens = clip.tokenize(text_descriptions).cuda()
```

```
[ ] plt.figure(figsize=(10, 8))

    for i, image in enumerate(original_images):
        plt.subplot(4, 4, 2 * i + 1)
        plt.imshow(image)
        plt.axis("off")

        plt.subplot(4, 4, 2 * i + 2)
        y = np.arange(top_probs.shape[-1])
        plt.grid()
        plt.barh(y, top_probs[i])
        plt.gca().invert_yaxis()
        plt.gca().set_axisbelow(True)
        plt.yticks(y, [cifar100.classes[index] for index in top_labels[i].numpy()])
        plt.xlabel("probability")

    plt.subplots_adjust(wspace=0.5)
    plt.show()
```

Observe the results and how close some of the image classifications are. Most of the results are very exceptional particularly how the VLM classified the horse gray mapped image as “cattle”.

Although, the page image classification is incorrectly labeled as a probably bed.


sweet_pepper
tiger
leopard
boy
mouse
0.0 0.1 0.2 0.3
motorcycle
bicycle
lawn_mower
pickup_truck
tractor
0.00 0.25 0.50 0.75 1.00
cattle
plain
camel
dinosaur
sweet_pepper
0.0 0.1 0.2 0.3
cup
bowl
plate
table
plain
0.00 0.25 0.50 0.75 1.00
rocket
skyscraper
clock
bus
woman
0.00 0.25 0.50 0.75 1.00
woman
girl
boy
rocket
plain
0.0 0.2 0.4 0.6 0.8
boy
man
plain
rocket
dinosaur
0.0 0.1 0.2 0.3
probability
Region-based segmentation
bridge
keyboard
bed
table
clock
0.00 0.05 0.10 0.15
probability

Debugging Steps Utilized

Debugging

Code Debugging Section for assigning root and data_dir file locations for images

```
[ ] # create a file folder location accessible via google drive
    # this will enable the use of local images
    from google.colab import drive
    drive.mount('/gdrive')
    # the project's folder
    %cd /gdrive/'My Drive'/Lab_Files
```

```
# Assign a root directory
root_dir = "/gdrive/MyDrive/"
data_dir = root_dir + 'Lab_Files'
print("base directory")
%ls {base_dir}
```

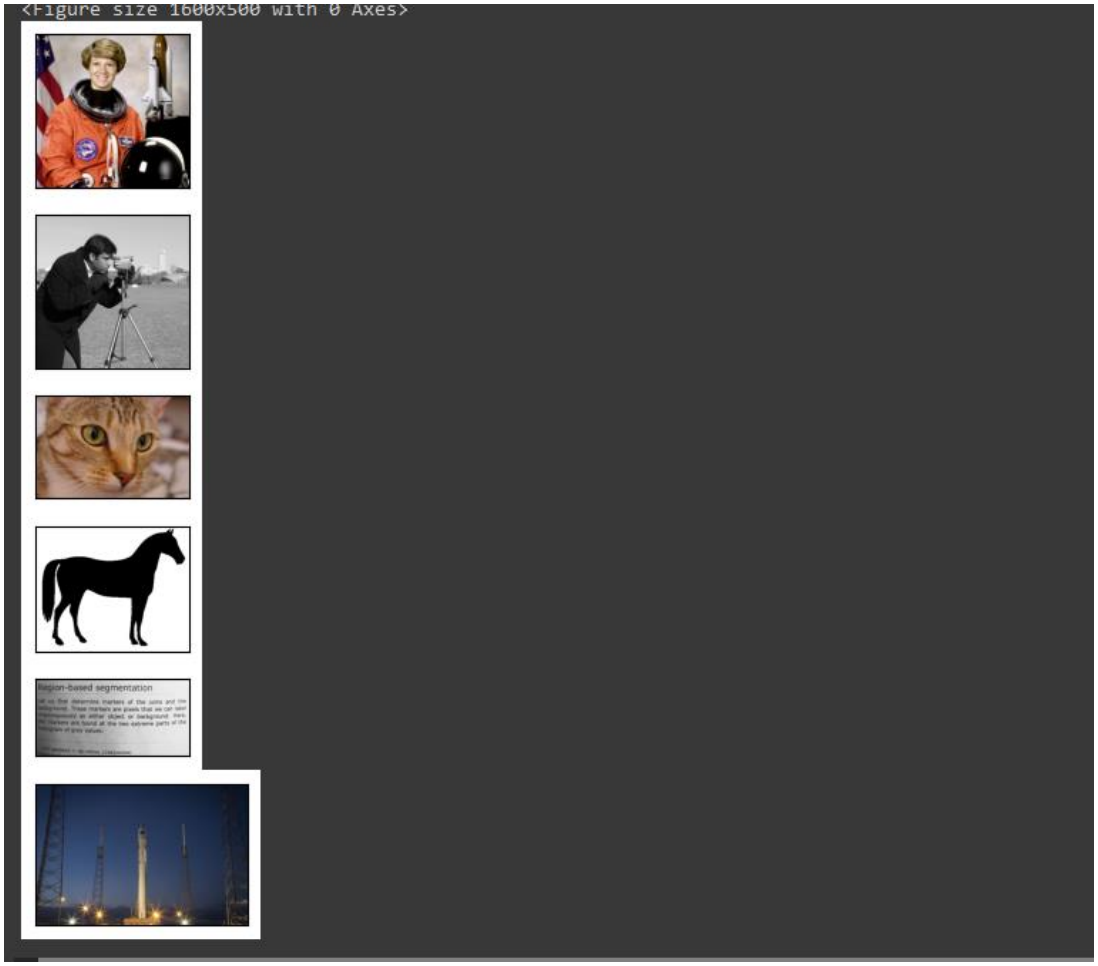
```
base directory
astronaut.png  chelsea.png  horse.png             page.png
camera.png     coffee.png   motorcycle_right.png  rocket.jpg
```

```
[ ] #Attempt to show all images in directory

    from skimage.io import imread
    # Get list of files in the directory
    for filename in os.listdir(data_dir):
      filepath = os.path.join(data_dir, filename)

      # Check if it's a file and then try to read it
      if os.path.isfile(filepath):
        name = os.path.splitext(filename)[0]
        if name not in descriptions:
            continue

        image = Image.open(os.path.join(filename)).convert("RGB")
        #I = imread(filepath)
        plt.imshow(image)
        plt.show()
```

✓ 10s completed at 4:52 PM

```
import matplotlib.pyplot as plt
import matplotlib
import numpy as np

from skimage import data

matplotlib.rcParams['font.size'] = 6
plt.figure(figsize=(16, 5))
images = (
    'astronaut',
    'camera',
    'cat',
    'horse',
    'page',
    'rocket',
)


for name in images:
    caller = getattr(data, name)
    image = caller()
    plt.figure()
    plt.title(name)
    if image.ndim == 2:
        plt.subplot(2, 4, len(images) + 1)
        plt.xticks([])
        plt.yticks([])
        plt.imshow(image, cmap=plt.cm.gray)
    else:
        plt.subplot(2, 4, len(images) + 1)
        plt.imshow(image)

        plt.xticks([])
        plt.yticks([])

#plt.show()
plt.tight_layout()
```

```
<Figure size 1600x500 with 0 Axes>
```



Conclusion

Overall, this exercise provides engineers with a comprehensive introduction and tutorial for deep learning multi-modal model development using *CLIP*. The steps introduced to integrated vision-language models which can be tuned for use in specific tasks and domains with computer vision and image recognition applications. Results of the lab also demonstrated the effectiveness of the model to discern images for classification identification. Moreover, with little or no fine-tuning, the model was able to exhibit exceptional performance with a zero-shot CLIP demonstration showing versatility with the ability to perform highly accurate image classification tasks without requiring specific fine-tuning or specialized training. This makes it a valuable tool for various applications, including image retrieval and captioning, image classification, image search, and text-to-image generation [1].

References

[1] Gohel, Hardik. Multi-modal Models. COSC 6370 – Advanced Topics in AI Module 1-8. https://hpedsi.uh.edu/people/hardik-gohelLinks to an external site.

[2] Rosidi, N. (n.d.). Multimodal Models Explained. Unlocking the Power of Multimodal Learning: Techniques, Challenges, and Applications. https://www.kdnuggets.com/2023/03/multimodal-models-explained.html

[3] Multimodal AI. (n.d.). Google Cloud. https://cloud.google.com/use-cases/multimodal-ai#features

[4] GeeksforGeeks. (2024, July 23). Bagging vs Boosting in Machine Learning. GeeksforGeeks. https://www.geeksforgeeks.org/bagging-vs-boosting-in-machine-learning/

[5] Aastha. (2024, July 19). Understanding multimodal AI: a new frontier in artificial intelligence. Silicon Valley Innovation Center. https://siliconvalley.center/blog/understanding-multimodal-ai-a-new-frontier-in-artificial-intelligence

[6] Openai. (n.d.). CLIP/notebooks/Interacting_with_CLIP.ipynb at main · openai/CLIP. GitHub. https://github.com/openai/CLIP/blob/main/notebooks/Interacting_with_CLIP.ipynb

[7] Google Colab. (n.d.). https://colab.research.google.com/github/openai/clip/blob/master/notebooks/Interacting_with_CLIP.ipynb#scrollTo=YPHN7PJgKOzb

[8] Lynne. (n.d.). How to setup Colab w/ Google Drive data hosting for Training/Retraining TF models. towardsdatascience.com. http://borg.csueastbay.edu/~grewe/CS663/Mat/TensorFlow/ColabRetrainSetupInstructions.pdf